DevDocs — Project Isolation & Project-Scoped RAG

1. Purpose

DevDocs can contain multiple software projects.

For example:

```
DevDocs
│
├── E-Commerce Application
│
├── Banking Application
│
└── Employee Management System
```

Each project has its own:

- Files
- File metadata
- Chunks
- Embeddings
- Conversations
- Knowledge

The knowledge of one project must not accidentally be used to answer questions about another project.

This concept is called:

> **Project isolation**

For RAG, this becomes:

> **Project-scoped retrieval**

---

2. The Core Problem

Suppose DevDocs contains two projects.

Project A — E-Commerce

```
backend/auth.py
backend/payment.py
```

```
frontend/App.jsx
README.md
```

Project B — Banking

```
backend/auth.py
backend/transactions.py
backend/accounts.py
README.md
```

Both projects may contain files with the same names.

Both may even contain similar concepts such as:

```
authentication
users
transactions
database
API
```

If DevDocs searches the entire vector database without knowing which project the user is currently working with, it could retrieve information from both projects.

That could produce an incorrect answer.

---

3. Project ID

The solution is to give every project a unique identifier.

Example:

```
E-Commerce Application
project_id = 101

Banking Application
project_id = 102
```

The `project_id` becomes the identity of the project's knowledge.

---

4. Project ID Propagation

The project ID should be carried throughout the DevDocs pipeline.

```
Project
   │
   │ project_id = 101
   ↓
Files
   │
   │ project_id = 101
   ↓
Parsed content
   │
   │ project_id = 101
   ↓
Chunks
   │
   │ project_id = 101
   ↓
Embeddings
   │
   │ project_id = 101
   ↓
Vector Database
```

This means that every piece of knowledge stored for a project can be traced back to that project.

---

5. File-Level Isolation

When a file is stored, its project association is recorded.

Example:

```
File
------------------------------
file_id:      501
project_id:   101
file_path:    backend/auth.py
file_hash:    abc123...
```

Another file from another project:

```
File
------------------------------
file_id:      801
project_id:   102
file_path:    backend/auth.py
file_hash:    xyz789...
```

Even though both files have the same path:

```
backend/auth.py
```

they belong to different projects because their `project_id` values are different.

---

6. Chunk-Level Isolation

During ingestion, a file is split into chunks.

For example:

```
backend/auth.py
       ↓
   Chunk 1
   Chunk 2
   Chunk 3
```

Each chunk should retain the project information.

Example:

```
{
  "chunk_id": 1001,
  "project_id": 101,
  "file_id": 501,
  "file_path": "backend/auth.py",
  "content": "..."
}
```

Another chunk from the Banking project:

```
{
  "chunk_id": 2001,
  "project_id": 102,
  "file_id": 801,
  "file_path": "backend/auth.py",
  "content": "..."
}
```

The vector database can therefore contain chunks from many projects while still knowing which project each chunk belongs to.

---

7. One Vector Database vs Multiple Vector Databases

We do not necessarily need a separate vector database for every project.

A single vector store can contain:

```
Vector Database
│
├── Project 101
│   ├── Chunk A
│   ├── Chunk B
│   └── Chunk C
│
├── Project 102
│   ├── Chunk D
│   ├── Chunk E
│   └── Chunk F
│
└── Project 103
    ├── Chunk G
    └── Chunk H
```

The important part is that every vector/chunk has metadata such as:

```
project_id
file_id
file_path
```

We then use that metadata during retrieval.

For DevDocs V1, the preferred approach is:

> **One vector store + project metadata filtering.**

---

8. User Opens a Project

Suppose the user opens:

```
E-Commerce Application
```

The frontend knows:

```
current_project_id = 101
```

The user then asks:

> How does authentication work?

The request sent to the backend can conceptually look like:

```
{
  "project_id": 101,
  "question": "How does authentication work?"
}
```

The backend now knows that the question belongs to:

```
Project 101
```

---

9. Project-Scoped RAG

This is where project isolation becomes directly connected to RAG.

The normal RAG process is:

```
User Question
      ↓
Embedding
      ↓
Similarity Search
      ↓
Relevant Chunks
      ↓
LLM
      ↓
Answer
```

DevDocs adds one important step:

```
User Question
      ↓
Current Project ID
      ↓
Embedding
      ↓
```

```
Filtered Similarity Search
      ↓
Relevant Chunks
      ↓
LLM
      ↓
Answer
```

The retrieval must be restricted to the current project.

---

10. Example of Project-Scoped Retrieval

Suppose the vector database contains:

```
Chunk 1
project_id = 101
content = "Authentication uses JWT..."

Chunk 2
project_id = 101
content = "Payments are processed using..."

Chunk 3
project_id = 102
content = "Authentication uses OAuth..."

Chunk 4
project_id = 102
content = "Bank transactions are stored..."
```

The user is currently inside:

```
Project 101
```

and asks:

> How does authentication work?

The vector search should conceptually apply:

```
project_id == 101
```

before selecting the most similar chunks.

Therefore:

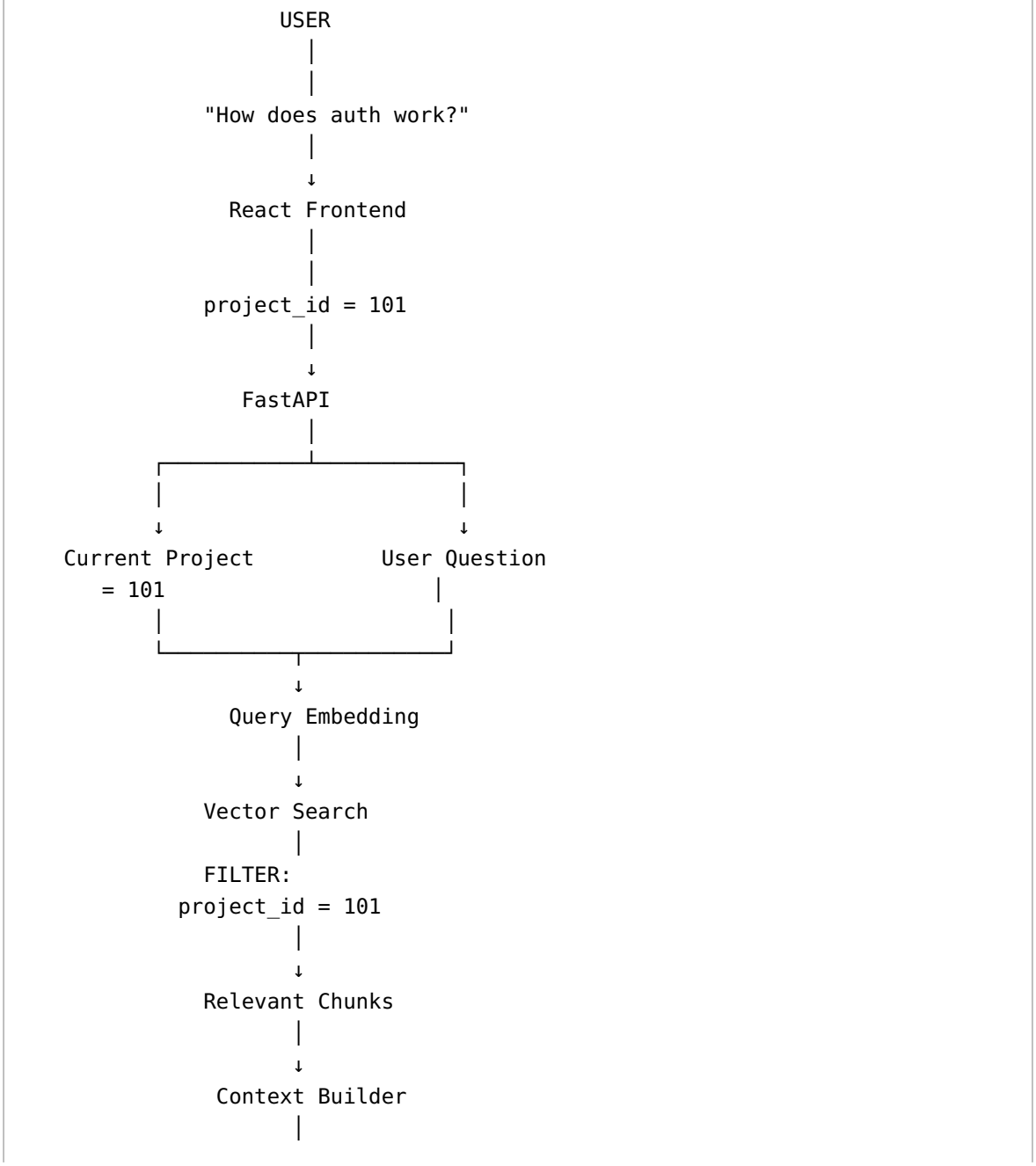
```
Chunk 1 → Eligible
Chunk 2 → Eligible
Chunk 3 → Excluded
Chunk 4 → Excluded
```

The RAG system can only retrieve knowledge belonging to Project 101.

---

11. Complete DevDocs RAG Flow

The complete flow becomes:



```
                ↓
               LLM
                │
                ↓
             Answer
```

The LLM itself does not need to understand project boundaries.

The **retrieval layer controls what information the LLM receives**.

---

12. Why the LLM Doesn't Need Project Isolation Logic

The LLM should not be responsible for deciding:

> "This information belongs to Project A, so I shouldn't use it."

That would be unreliable.

Instead:

```
Vector Database
      ↓
Retrieval Layer
      ↓
Only Project A chunks
      ↓
LLM
```

The LLM never receives Project B's chunks in the first place.

This is a much safer architecture.

---

13. Conversation Isolation

The same principle applies to conversations.

A conversation should belong to a project.

Example:

```
Conversation
----------------------------
```

```
conversation_id: 50
project_id:      101
```

Another conversation:

```
Conversation
----------------------------
conversation_id: 80
project_id:      102
```

Therefore, when the user opens Project 101, DevDocs loads conversations belonging to:

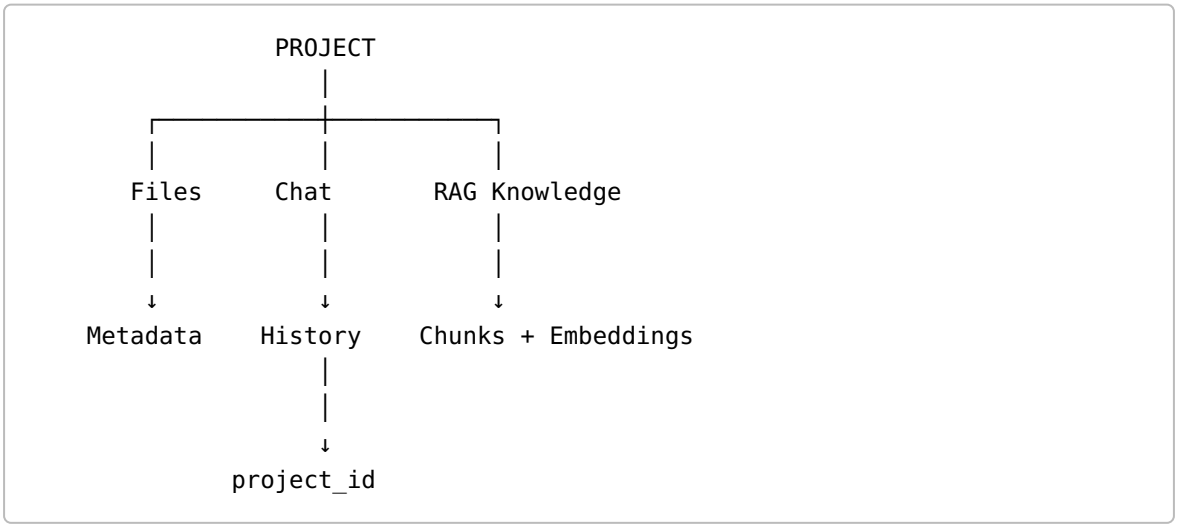
```
project_id = 101
```

Project 102's conversation history is not included.

---

14. Project Context

The current project therefore becomes the central context for the entire application.



Everything is connected through the project ID.

---

15. Security / Correctness Boundary

Project isolation is not only a UI feature.

It is a **backend correctness boundary**.

The backend should never simply trust:

```
"Give me all chunks similar to this question."
```

It should conceptually perform:

```
"Give me chunks similar to this question
WHERE project_id = the current project."
```

This prevents accidental cross-project retrieval.

---

16. Example: Switching Projects

Suppose the user is currently viewing:

```
E-Commerce
project_id = 101
```

They ask:

> How is authentication implemented?

DevDocs retrieves only:

```
project_id = 101
```

Now the user switches to:

```
Banking
project_id = 102
```

The next question:

> How is authentication implemented?

uses:

```
project_id = 102
```

The same question can therefore produce a completely different answer because the **knowledge boundary changed**.

---

17. Project Isolation Across the Ingestion Pipeline

Isolation starts before RAG.

When a project is uploaded:

```
Project 101
     ↓
File Discovery
     ↓
File Metadata
     ↓
Parsing
     ↓
Chunking
     ↓
Embedding
     ↓
Vector Database
```

At each stage, the project identity is preserved.

For example:

```
File
   ↓
project_id = 101

Chunk
   ↓
project_id = 101

Embedding
   ↓
project_id = 101
```

Therefore, when retrieval happens later, the system can filter using the same project ID.

---

18. Relationship With Incremental Ingestion

This also connects to the change-detection system.

Suppose:

```
Project 101
```

contains:

```
backend/auth.py
backend/payment.py
README.md
```

During re-upload:

```
backend/auth.py       → modified
backend/payment.py    → unchanged
README.md             → unchanged
```

Only the modified file needs to be reprocessed.

Its new chunks still retain:

```
project_id = 101
```

The project boundary therefore remains intact even during incremental ingestion.

---

19. Important Mental Model

Think of `project_id` as a **label that travels with the project's knowledge**.

```
Project
   ↓
project_id
   ↓
Files
   ↓
Chunks
   ↓
Embeddings
   ↓
Vector Database
   ↓
Filtered Retrieval
   ↓
LLM
```

The vector database may contain knowledge from 100 projects.

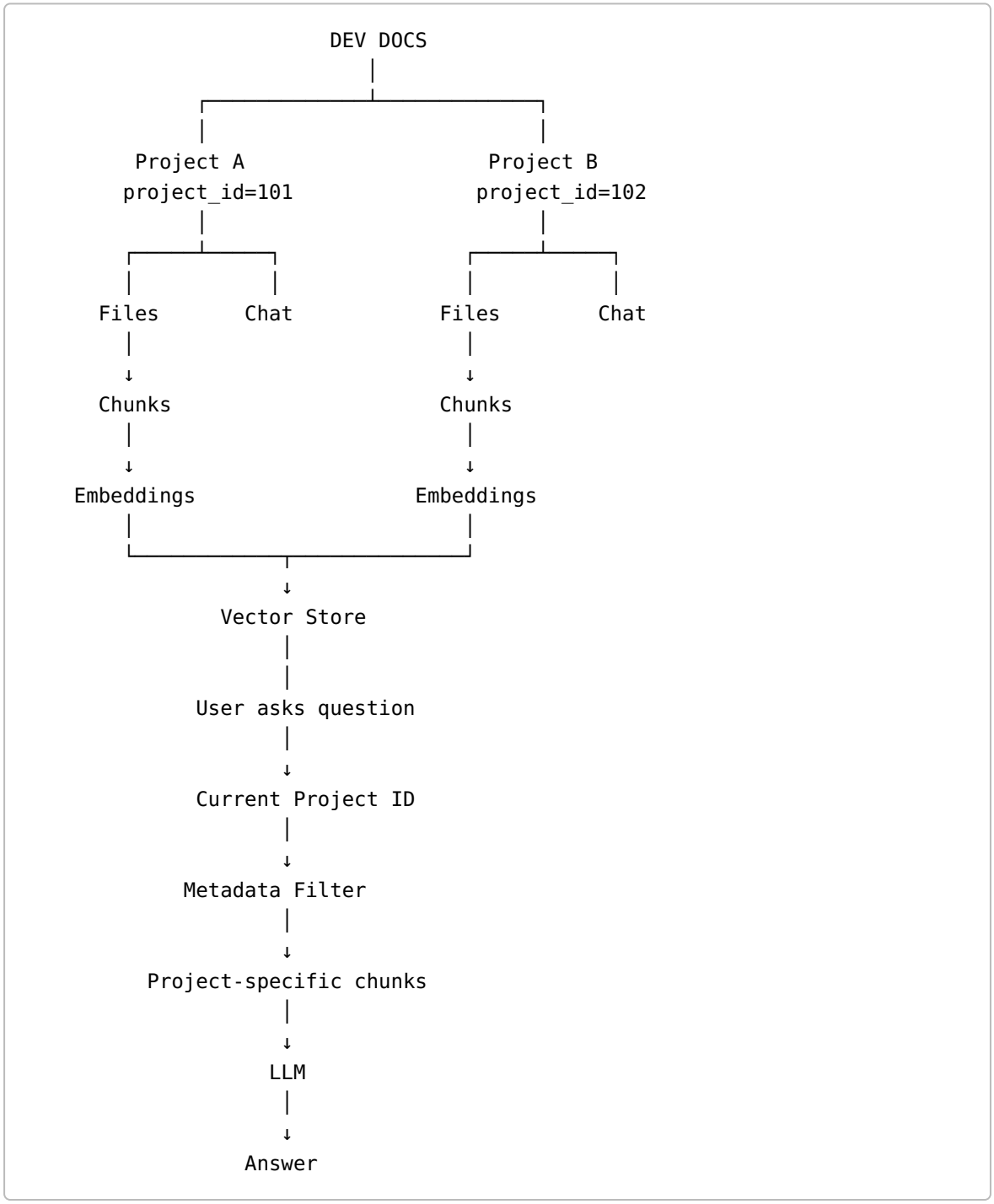
That is not a problem.

The problem would only occur if we retrieved from those 100 projects without filtering.

---

20. Final Architecture

The relevant part of DevDocs can therefore be visualized as:



---

21. Core Principle

The most important principle for DevDocs is:

> **The LLM should never be responsible for deciding which project knowledge it is allowed to see.**

Instead:

> **The application determines the current project, the retrieval layer filters by that project, and only the filtered context is provided to the LLM.**

This is how DevDocs keeps the knowledge of one project isolated from another while still allowing all projects to share the same underlying RAG infrastructure.